DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: **04-11-2026**

NAME: T.Tharun Sai

REG NO: 192424198

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c ___d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding

token-level language identification, or falling back to stemming for out-of-lexicon tokens.

- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

1.1 Problem definition

A hospital stores large numbers of clinical notes as free text. A literal search for “fracture” may miss “fractures”, “fractured” and “fracturing”. Dates, patient identifiers, drug strengths and measurements may also be written in different formats. The objective is to create a preprocessing layer that converts inconsistent text into searchable tokens and canonical forms.

1.2 Levels of linguistic analysis

Level	Meaning	Hospital example and relevance
Phonology	Sound patterns of a language.	Not central for clean typed English records. It becomes relevant indirectly when OCR or speech input creates sound-like spelling errors.
Morphology	Structure of words and their morphemes.	Essential: fracture/fractures and operate/operated must be related.
Syntax	How words combine into phrases and sentences.	Useful for interpreting phrases such as “no evidence of infection”; basic search normalisation does not need a full parser.
Semantics	Meaning of words and expressions.	Important for distinguishing clinically related terms and avoiding misleading matches such as renal disease versus renal failure.
Pragmatics	Meaning in a particular situation or speaker intention.	Useful for interpreting instructions and context, but not required for basic entity recognition.
Discourse	Connections across sentences and the whole document.	Useful for resolving references and timelines, but beyond the initial finite-state layer.

1.3 Model families

Model family	Best hospital task	Why it fits / why alternatives are weaker
State machines	Validate fixed formats such as patient IDs, dates and structured codes.	Transitions give exact acceptance rules. A probabilistic model is unnecessary for a strict format; a general rule list alone does not naturally show state-by-state

		validation.
Formal rule systems	Apply spelling rules and morphological transformations.	Rules are transparent and maintainable. A state machine alone does not express all linguistic rewrite conditions conveniently; a probabilistic model needs labelled examples.
Probabilistic models	Rank uncertain OCR corrections or choose between ambiguous analyses.	They can represent uncertainty. Exact regular expressions are too strict for noisy input, while hand-written rules may not cover every context.

1.4 Ambiguity in clinical records

- Lexical ambiguity: “cold” can refer to low temperature or a common cold.
- Morphological ambiguity: “patients” can be analysed as patient + noun + plural, while some surface forms may have more than one grammatical analysis.
- Segmentation ambiguity: “500mg” may be treated as one token or as the number “500” followed by the unit “mg”.
- Abbreviation ambiguity: “MS” may mean multiple sclerosis, morphine sulfate or another local term depending on context.

1.5 Search and resources

Automata use state-space traversal: the input is read symbol by symbol and the machine changes state. Regular-expression engines may use backtracking to explore alternative matches. Time depends on input length and pattern structure; memory is used for states, transitions and match bookkeeping. Annotated data is needed for evaluation and for resolving uncertain cases. Expert effort is required to define clinical vocabulary, abbreviations and safe normalisation rules.

1.6 Proposed pipeline

Clinical records
|
v
[Input validation] -- regex / finite automata
|
v
[Tokenisation] -- regular expression + maximal munch
|
v
[Normalisation] -- case folding, abbreviation and unit rules
|
v
[Morphological analysis] -- lexicon + finite-state transducer
|
v
[Search index] -- canonical terms and entity fields
|
v

[Query processing and retrieval]
|
v
Ranked relevant hospital records

Input: raw clinical notes and clinician queries. Output: validated entities, normalised tokens, morphological analyses and a searchable index.

TASK 2 – Basic Regular Expression Patterns

2.1 Basic operators

Operator	Meaning	Example
Concatenation	Items occur in sequence.	AB matches A followed by B.
Disjunction |	Either alternative is accepted.	mg|g matches mg or g.
Kleene star *	Zero or more repetitions.	[0-9]* matches any number of digits.
Plus +	One or more repetitions.	[0-9]+ matches one or more digits.
Optional ?	Zero or one occurrence.	-? allows an optional minus sign.
Character class []	One character from a set.	[A-Z] matches an uppercase letter.
Range	A continuous character interval.	[0-9] matches a digit.
Negated class [^]	Any character except listed characters.	[^\s]+ matches a non-whitespace run.
Anchors ^ and $	Beginning and end of a string/line.	^[0-9]{4}$ matches exactly four digits.
Counters {m,n}	A bounded number of repetitions.	[0-9]{2,4} matches 2 to 4 digits.

2.2 Twelve clinical entity patterns

Entity	Python regular expression	Example match	Must not match
Patient identifier	\bPT[0-9]{6}\b	PT123456	Reject PT12345 and PT1234567.
Admission date	\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b	04/11/2026	Reject impossible dates such as 39/19/2026; calendar validation is a second step.
Discharge date	\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b	06-11-2026	Reject missing day, month or year.
Drug + strength	\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s?(?:mg|mcg|g|ml|mL)\b	Paracetamol 500mg	Reject a drug name with no strength or unsupported unit.
Dosage frequency	\b(?:OD|BD|TDS|QID|once daily|twice daily|every\s+[0-9]+\s+hours)\b	twice daily	Reject unrelated words such as “daily review”.
Lab value + unit	\b[A-Za-z][A-Za-z-]*\s*[:=]?\s*[0-9]+(?:\.[0-9]+)?\s?(?:mg/dL|mmol/L|g/dL|U/L)\b	Glucose: 110 mg/dL	Reject a value without a recognised unit.

Blood pressure	\b[0-9]{2,3}\s*/\s*[0-9]{2,3}\s?mmHg\b	120/80 mmHg	Reject text without mmHg or with missing pressure values.
Temperature	\b(?:3[4-9]|4[0-2])(?:\.[0-9])?\s?°?C\b	37.2 C	Reject values outside the configured screening range.
ICD-style code	\b[A-Z][0-9]{2}(?:\.[0-9A-Z]{1,4})?\b	S72.001A	Reject a code with no leading letter or too many characters.
Ward and bed	\bWard\s+[A-Z0-9-]+\s*,?\s*Bed\s+[A-Z0-9-]+\b	Ward A, Bed 12	Reject a ward label without a bed identifier.
Contact number	\b(?:\+91[-]?)?[6-9][0-9]{9}\b	9876543210	Reject fewer than ten digits or an invalid starting digit.
Age	\b(?:[1-9][0-9]?|1[01][0-9]|120)\s?(?:years?|yrs?)\b	45 years	Reject negative ages and values above 120.

The expressions are screening patterns, not a substitute for clinical validation. Dates need calendar checks, temperature ranges need local policy, and ICD patterns should be aligned to the code-set version used by the hospital.

2.3 Greedy and non-greedy matching

```
import re
text = "Give 500mg after breakfast; repeat 250mg at night."
print(re.findall(r".*mg", text))   # greedy: longest span ending at the last mg
print(re.findall(r".*?mg", text))  # non-greedy: shortest spans ending at each mg
```

A greedy quantifier consumes as much text as possible while allowing the rest of the pattern to match. A non-greedy quantifier consumes as little as possible. For entity extraction, a focused expression such as [0-9]+\s?mg is safer than a broad dot-star expression.

2.4 Precedence and grouping

Concatenation binds more tightly than alternation. Therefore, ab|cd means either “ab” or “cd”. The expression ^ab|cd$ is not equivalent to ^(ab|cd)$: the first can match a string beginning with ab or a string ending with cd. Grouping makes the intended alternatives explicit.

2.5 Evaluation method

For each entity, compare the pattern output with manually labelled spans. A true positive (TP) is a correct extracted entity; a false positive (FP) is an incorrect extraction; a false negative (FN) is a missed entity. Precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2PR/(P+R). Record OCR errors, spacing variation, unseen abbreviations and invalid values as separate error causes. Do not report corpus-wide scores until the actual corpus has been annotated.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

3.1 Token definition and tokenizer

A token is a meaningful searchable unit. Hyphenated clinical terms remain one token; abbreviations with internal full stops remain together; decimal values remain intact; and a number with a recognised unit is treated as one measurement token.

```
TOKEN_RE = re.compile(
    r"(?:[A-Za-z]\.){2,}|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"[0-9]+(?:\.[0-9]+)?(?:mg|mcg|g|ml|mL|mmHg|°C)?|"
    r"[^\w\s]",
    re.IGNORECASE
)
```

3.2 Maximal-munch examples

Input	Whitespace split	Maximal-munch tokenisation
BP was 120/80 mmHg.	['BP','was','120/80','mmHg.']	['BP','was','120','/','80','mmHg','.']
Give 500mg twice-daily.	['Give','500mg','twice-daily.']	['Give','500mg','twice-daily','.']
Dr. Rao reviewed 37.5°C.	['Dr.','Rao','reviewed','37.5°C.']	['Dr.','Rao','reviewed','37.5°C','.']

Maximal munch selects the longest valid token at the current position. A production tokenizer should use an ordered set of token rules so that recognised abbreviations and measurement units are preferred over shorter fragments.

3.3 Normalisation rules

- Fold English text to lowercase for case-insensitive search while retaining the original text for display.
- Standardise spaces around punctuation and measurements.
- Convert date separators to a common representation after validating the date.
- Convert equivalent units only when the unit and conversion are unambiguous.
- Expand known abbreviations using a maintained, context-aware dictionary.

3.4 Abbreviation dictionary (30 examples)

Abbreviation	Expansion
BP	blood pressure
HR	heart rate
RR	respiratory rate
Temp	temperature
SpO2	oxygen saturation
SOB	shortness of breath
DOB	date of birth

DM	diabetes mellitus
HTN	hypertension
CAD	coronary artery disease
CVA	cerebrovascular accident
MI	myocardial infarction
UTI	urinary tract infection
COPD	chronic obstructive pulmonary disease
CKD	chronic kidney disease
CNS	central nervous system
IV	intravenous
IM	intramuscular
PO	by mouth
OD	once daily
BD	twice daily
TDS	three times daily
QID	four times daily
PRN	as needed
NPO	nothing by mouth
OPD	outpatient department
ICU	intensive care unit
ER	emergency room
CBC	complete blood count
RBS	random blood sugar

Abbreviations can be ambiguous. Expansion should depend on a clinical dictionary and, where needed, nearby words or department context. The original abbreviation must be preserved in the source record.

3.5 Before-and-after record

Synthetic source record: “PT123456 admitted on 04/11/2026 with fever. BP 120/80 mmHg, Temp 37.2C. Rx: Paracetamol 500mg BD. Discharged on 06-11-2026.”

Normalised search view: “pt123456 admitted on 2026-11-04 with fever. blood pressure 120/80 mmhg, temperature 37.2 c. prescription: paracetamol 500 mg twice daily. discharged on 2026-11-06.”

3.6 Irreversible decisions and limits

- Lowercasing removes original capitalisation; usually acceptable for search, but not for names or abbreviations where case carries meaning.
- Removing punctuation may destroy decimal, code or sentence boundaries; preserve the original string.
- Expanding an ambiguous abbreviation to one meaning can introduce a false interpretation; retain the original and record the chosen expansion.

A basic regular-expression layer cannot reliably match arbitrarily nested balanced parentheses. A finite automaton has only a finite number of states and cannot remember an unlimited nesting depth. A parser or stack-based machine is needed for that requirement.

TASK 4 – Finite-State Automata: Design, Non-determinism and Equivalence

4.1 Formal definition

A deterministic finite automaton (DFA) is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators

The following state descriptions provide compact designs. Each DFA has a start state, accepting state(s), and a dead state for invalid transitions. In an implementation, character classes such as digit, uppercase letter and separator are treated as input-symbol groups.

Validator	Transition outline	Acceptance condition
Patient ID PT + 6 digits	q0 --P--> q1 --T--> q2 --digit--> q3 --digit--> q4 --digit--> q5 --digit--> q6 --digit--> q7 --digit--> q8*	Accept only PT followed by exactly six digits.
Age: 1–120 years	q0 --digit--> q1; q1 branches by numeric range; qA* after valid 1–120 and optional unit suffix	Numeric range validation is easiest with a small DFA plus a final numeric check.
Blood pressure: 2–3 digits / 2–3 digits mmHg	q0 --digit--> q1 --digit--> q2; optional digit; '/' -> q3; 2–3 digits -> q4; suffix mmHg -> qA*	Reject missing numerator, denominator or unit.
ICD-style code	q0 --A-Z--> q1 --digit--> q2 --digit--> q3; optional '.' and 1–4 alphanumeric characters	Reject wrong initial character and excessive code length.

4.3 NFA with optional components

Consider a patient identifier that may contain an optional hyphen after the prefix: PT123456 or PT-123456. An NFA can branch after reading PT: one branch consumes a hyphen and the other skips it using an epsilon transition.

NFA states:
q0 --P--> q1 --T--> q2
q2 --ε--> q3 (no hyphen)
q2 --'-'--> q3 (hyphen present)
q3 --digit--> q4 --digit--> q5 --digit--> q6
--digit--> q7 --digit--> q8 --digit--> q9*
q9 is the only accepting state.

Accepted trace for PT-123456: {q0} --P--> {q1} --T--> {q2} --'-'--> {q3} --1--> {q4} --2--> {q5} --3--> {q6} --4--> {q7} --5--> {q8} --6--> {q9}; accepted. Rejected example PT-12345 ends in q8, which is not accepting.

4.4 Subset construction and minimisation

Subset construction converts an NFA into a DFA. Each DFA state represents a set of NFA states. Begin with the epsilon-closure of the NFA start state. For each input symbol, collect all reachable states and take their epsilon-closure. Continue until no new subsets appear. The accepting DFA subsets are those containing q9.

DFA subset	Input digit	Input hyphen	Accepting?
{q0}	∅	∅	No
{q1}	∅	∅	No
{q2}	∅	{q3}	No
{q3}	{q4}	—	No
{q4}	{q5}	∅	No
{q5}	{q6}	∅	No
{q6}	{q7}	∅	No
{q7}	{q8}	∅	No
{q8}	{q9}	∅	No
{q9}	∅	∅	Yes

For this simple pattern the optional hyphen creates two paths that merge at q3. Minimisation merges equivalent states only when their future acceptance behaviour is identical. The exact state count depends on whether digit classes and the dead state are represented explicitly; document the chosen alphabet partition in the final implementation.

4.5 Thompson construction and equivalence

Thompson's construction converts a regular expression into an NFA by creating small machines for symbols, concatenation, alternation and Kleene closure, joined with epsilon transitions. Conversely, state elimination can derive a regular expression from an automaton. The resulting automaton and expression are equivalent when they accept exactly the same set of strings.

4.6 Pumping lemma

The pumping lemma states that for every regular language there is a pumping length p such that every string s in the language with length at least p can be split as $s = xyz$, where $|xy| \leq p$, $|y| > 0$ and xy^iz remains in the language for every $i \geq 0$.

The language $L = \{ '('^n ')'^n : n \geq 0 \}$ is not regular. Choose $s = '('^p ')'^p$. Since $|xy| \leq p$, the non-empty y contains only opening parentheses. Pumping y down (i = 0) leaves fewer opening than closing parentheses, so the result is not in L. This shows why a basic regular-expression/finite-automaton layer cannot validate unlimited balanced nesting.

TASK 5 – Inflectional Morphology

5.1 Core terms

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme placed before or after a stem. Inflection changes grammatical form without normally changing the word class or core dictionary meaning. A paradigm is the set of forms associated with a lexeme.

Derivation creates a new lexeme and may change word class or meaning. Inflection is generally more productive and occurs outside derivational material in many English forms, although the exact order depends on the word.

5.2 Inflectional paradigms

Paradigm	Examples	Function
Noun number	fracture/fractures; kidney/kidneys	Singular and plural.
Verb tense/aspect	treat/treats/treated/treating	Tense, agreement and progressive aspect.
Subject agreement	patient reports; nurse records	Third-person singular present adds -s.
Adjective comparison	high/higher/highest; mild/milder/mildest	Comparative and superlative forms.

Lexeme	Inflectional forms
fracture	fracture, fractures
patient	patient, patients
treat	treat, treats, treated, treating
monitor	monitor, monitors, monitored, monitoring
high	high, higher, highest

5.3 English spelling rules

Rule — Description — Example — Possible over-generation

• E-deletion — Drop final silent e before a vowel-initial suffix where required. — operate → operating — see → seeing does not follow the same simple deletion pattern.

• Consonant doubling — Double a final consonant in a stressed short-vowel pattern before a vowel suffix. — admit → admitted — visit → visiting; doubling it would be wrong.

• Y-to-i — Change consonant + y to i before many suffixes. — injury → injuries — play → plays retains y before -s.

• Sibilant -es — Add -es after s, x, z, ch or sh sounds. — dose → doses — The rule must not add -es to every noun.

5.4 Irregular forms

Base — Irregular form — Reason

• be — was, were, been — Suppletive forms; cannot be generated by a simple suffix rule.

• go — went, gone — Irregular stem alternation.

• child — children — Irregular plural.

• foot — feet — Vowel change.

• tooth — teeth — Vowel change.

• diagnosis — diagnoses — Greek-derived plural pattern.

• analysis — analyses — Greek-derived plural pattern.

Irregular forms should be listed in the lexicon because their surface form cannot be predicted reliably from the productive regular rules alone.

5.5 Measuring inflection

Type-to-lemma ratio = number of distinct surface word types / number of distinct lemmas. A lower ratio after lemmatisation indicates that multiple surface forms have been grouped under common lemmas. To measure query failures explained by inflection, label each failed query, then count the cases where the relevant record contains an inflected variant of the query lemma. The percentage is (inflection-caused failures / all query failures) × 100. Actual corpus counts must be inserted after annotation.

TASK 6 – Derivational Morphology

6.1 Definition and affix inventory

Derivation forms a new lexeme by attaching a prefix or suffix to a root or stem. It is harder than inflection because it can change meaning and word class, and derived meanings are not always predictable from their parts.

Affix	Position	Typical function	Example
-itis	Suffix	Inflammation; noun-forming	tonsillitis
-ectomy	Suffix	Surgical removal; noun-forming	nephrectomy
-ology	Suffix	Field or study; noun-forming	cardiology
-osis	Suffix	Condition or process; noun-forming	fibrosis
-aemia	Suffix	Blood-related condition; noun-forming	anaemia
-ation	Suffix	Action or result; noun-forming	inflammation
-able	Suffix	Capable of; adjective-forming	treatable
-ive	Suffix	Having a tendency/function; adjective-forming	protective
nephro-	Prefix	Kidney-related modifier	nephrology
cardio-	Prefix	Heart-related modifier	cardiovascular
hyper-	Prefix	Excessive/high	hypertension
hypo-	Prefix	Low/under	hypoglycaemia
re-	Prefix	Again/back	reassessment
un-	Prefix	Negation/reversal	untreated
-al	Suffix	Relating to; adjective-forming	renal

6.2 Derivational families

Root/family	Related forms	Notes
nephr-	nephron, nephritis, nephrology, nephrectomy	Kidney-related terms; suffixes specify condition, field or procedure.

cardi-	cardiac, cardiology, cardiologist, cardiomyopathy	Root combines with several derivational elements.
fract-	fracture, fractured, refracture	Re- attaches before the base; -ed is inflectional.
infect-	infect, infection, infectious, disinfect	Prefix and suffix combinations change meaning and category.
operat-	operate, operation, operative, postoperative	Pre-/suffixes build related clinical vocabulary.
treat-	treat, treatment, treatable, untreated	Derivational and inflectional processes must be separated.

Possible order: un- + treat + -ed → untreated (the negative prefix attaches to the lexical base and the past participle is inflectional). Another example is re- + assess + -ment → reassessment. Impossible or ill-formed orders include *-ectomy + nephro- as a suffix-first construction, and *treat-able-un where the negative prefix is placed after the suffix.

6.3 Non-compositionality and search

A derived word may have a specialised meaning that cannot be calculated by simply combining its parts. “Nephrectomy” means surgical removal of a kidney, not merely a generic kidney-related event. “Anaemia” is a specific clinical condition. Aggressive stemming may connect terms that share letters but differ clinically, creating misleading retrieval.

6.4 Productivity and conflation

Affix productivity should be measured by counting distinct word types and total token occurrences for each affix in the corpus. An affix found in many newly formed words is likely productive; one found only in a small fixed set may be fossilised. The corpus counts must be added after running the analysis.

Conflation decision — Examples — Effect

• Helpful — fracture / fractures; nephrectomy / nephrectomies — Search can retrieve plural or inflected variants.

• Potentially helpful with context — infect / infection; operate / operation — Related concepts may be useful, but their grammatical and clinical roles differ.

• Misleading — renal failure / renal function — Shared root does not mean the concepts are interchangeable.

• Misleading — hypertension / hypotension — The prefixes hyper- and hypo- reverse the clinical meaning.

Recommendation: conflate inflectional variants by default, but treat derivational relatives as related search expansions rather than identical terms. Preserve negation and clinically meaningful prefixes.

TASK 7 – Finite-State Morphological Parsing

7.1 Finite-state transducer

A finite-state transducer (FST) is an automaton with an input tape and an output tape. In morphology, the surface tape contains the observed word and the lexical tape contains the lemma plus grammatical features. The transducer can analyse surface forms and generate surface forms from lexical representations.

Example: surface “fractures” is analysed as lexical “fracture +N +PL”. The input tape consumes the letters f-r-a-c-t-u-r-e-s; the output tape emits the lexical stem “fracture” and the features +N +PL.

7.2 Lexicon and continuation classes

Lexicon class	Example entries	Continuation
NOUN	fracture, patient, kidney	Noun inflection: +SG, +PL
VERB	treat, monitor, admit	Verb inflection: +BASE, +3SG, +PAST, +PROG
ADJECTIVE	high, mild, severe	Comparison: +POS, +COMP, +SUP
IRREGULAR	be/was/been; child/children	Explicit surface-to-lexical pairs

7.3 Hand-traced analyses

Surface form	Lexical analysis	Feature interpretation
fractures	fracture +N +PL	Plural noun
patients	patient +N +PL	Plural noun
treated	treat +V +PAST	Past-tense verb or participle depending on context
monitoring	monitor +V +PROG	Progressive/gerund form
higher	high +ADJ +COMP	Comparative adjective

A hand trace follows the path through the lexicon stem states, then the relevant suffix path, and ends in a final analysis state. The surface letters are consumed while lexical symbols and features are emitted.

7.4 Generation direction

The same relation can be traversed in reverse: lexical input fracture +N +PL generates the surface form “fractures”. This is not a second grammar; it is the same set of lexical/surface correspondences used in the opposite direction.

7.5 Ambiguity

- “treated” may be a past-tense verb or a past participle; the surface form alone does not determine its syntactic role.
- “monitoring” may be a verb form or a noun/gerund. A downstream sentence-level component can use context to rank the analyses.

The analyser should return all valid analyses. A later context-aware component selects or ranks them; the morphological layer should not silently discard alternatives.

TASK 8 – Orthographic Rules and Two-Level Morphology

8.1 Why a lexicon alone is insufficient

A lexicon that simply concatenates stems and suffixes may produce incorrect spellings. Examples include admit + ed → *admited instead of admitted; injury + s → *injurys instead of injuries; and operate + ing → *operateing instead of operating.

8.2 Orthographic rewrite rules

Rule	Rewrite notation (informal)	Example
Final e deletion	e → 0 / stem __+V-suffix	operate + ing → operating
Consonant doubling	C → CC / stressed short vowel C __+ed	admit + ed → admitted
Consonant-y plural	y → i / consonant __+es	injury + es → injuries
Sibilant plural	0 → e / sibilant __+s	dose + s → doses
Plural after ch/sh/x	0 → e / (ch|sh|x) __+s	match + s → matches

Here 0 denotes deletion or insertion of no lexical symbol, depending on the rewrite convention. A formal implementation must define the boundary symbols and the exact contexts.

8.3 Cascade and two-level morphology

Lexical representation
|
v
Lexicon FST
|
v
Orthographic rule FST 1: e-deletion
|
v
Orthographic rule FST 2: consonant doubling
|
v
Orthographic rule FST 3: y-to-i
|
v
Orthographic rule FST 4: sibilant -es
|
v
Surface word

A sequential cascade composes rule transducers in a defined order. This is easy to inspect and debug, but ordering dependencies can cause one rule to affect another. The two-level (Koskenniemi) approach constrains lexical and surface symbol pairs in parallel, avoiding a strict sequence of rewrite passes. It can be more elegant for interacting spelling rules, but its rule notation and implementation are more demanding.

8.4 Test set

Lexical input	Expected surface	Expected test
operate + PROG	operating	Pass
admit + PAST	admitted	Pass
injury + PL	injuries	Pass
dose + PL	doses	Pass
match + PL	matches	Pass
treat + PAST	treated	Pass
monitor + PROG	monitoring	Pass
fracture + PL	fractures	Pass
patient + PL	patients	Pass
kidney + PL	kidneys	Pass
study + PL	studies	Pass
play + 3SG	plays	Pass
visit + PROG	visiting	Pass
stop + PAST	stopped	Pass
fix + PL	fixes	Pass
box + PL	boxes	Pass
church + PL	churches	Pass
carry + PAST	carried	Pass
see + PROG	seeing	Pass
be + PAST	was	Lexicon exception

This is a designed test list, not a measured run. In the implementation, report actual success/failure counts and explain failures by rule coverage, irregularity or incorrect boundary conditions.

TASK 9 – The Porter Stemmer

9.1 Algorithm concepts

The Porter stemmer is a rule-based suffix-stripping algorithm. It does not require a full dictionary. It uses consonant/vowel patterns and the measure m, which counts vowel-consonant sequences in the stem. The conditions *v, *d and *o refer to the presence of a vowel in the stem, a double consonant at the end, and a consonant-vowel-consonant ending where the final consonant is not w, x or y.

9.2 Illustrative clinical examples

Word	Possible Porter stem	Comment
fractures	fractur	Plural suffix removed.
patients	patient	Plural suffix removed.
treated	treat	Past-tense suffix removed.
monitoring	monitor	Progressive suffix removed.
relational	relat	Derivational suffix reduction.
sensitivities	sensit	Suffix sequence is reduced; result is not necessarily a dictionary lemma.

The exact intermediate sequence depends on the full Porter rule ordering and should be verified by the implementation. A stem is a search-oriented string, not necessarily a valid English word.

9.3 Five-step outline

Step	Main operation	Example effect
1a	Remove plural endings.	caresses → caress; ponies → poni
1b	Remove -ed/-ing under conditions and adjust the stem.	agreed → agree; disabled → disable
1c	Change terminal y to i when the stem contains a vowel.	happy → happi
2	Replace longer derivational suffixes with shorter forms.	relational → relate (subject to rule conditions)
3	Further derivational suffix replacement.	conditional → condition (rule-dependent)
4	Remove selected suffixes when measure m is sufficiently large.	removal of -ment, -ness, -able in qualifying stems
5	Remove final e or reduce final double l under conditions.	probate → probat; controll → control

The original Porter algorithm is commonly described in five broad steps, with some descriptions grouping the suffix stages differently. Use the exact NLTK implementation version for reproducible outputs and include the version in the report.

9.4 Stemming versus lemmatisation

Method — Output for fractures — Meaning

• No normalisation — fractures — Surface word remains unchanged.

• Porter stemming — fractur — Suffix-based search stem.

• Morphological lemmatisation — fracture +N +PL → fracture — Dictionary/grammar-based lemma and features.

Stemming is fast and lexicon-free but can over-stem unrelated terms or under-stem related terms. Lemmatisation is more linguistically accurate but needs a lexicon, affix rules and maintenance. To quantify agreement, compare both outputs for the same 200 hand-annotated forms and count exact matches and divergences.

9.5 Retrieval evaluation

Run the 40 clinician queries in three conditions: (1) no stemming, (2) Porter stemming, and (3) full morphological analysis. For each query, label relevant records and calculate precision, recall and F1. Aggregate macro-averaged scores across queries and also report per-query variation. Do not invent numerical results; insert values from the completed experiment.

Condition — Precision — Recall — F1 — Expected behaviour

• No stemming — Measure — Measure — Measure — High precision for exact forms; may miss variants.

• Porter stemmer — Measure — Measure — Measure — May improve recall; risk of over-stemming.

• Full morphology — Measure — Measure — Measure — Can use grammatical features and irregular forms; higher resource effort.

TASK 10 – Implement and Test the Pipeline

10.1 Python implementation

The following compact implementation demonstrates tokenisation, entity extraction, normalisation, a simple morphology lookup and Porter stemming. Extend the dictionaries and patterns using the documented corpus.

```
import re
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
    r"(?:[A-Za-z]\.){2,}|[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"[0-9]+(?:\.[0-9]+)?(?:mg|mcg|g|ml|mL|mmHg|°C)?|[^\w\s]",
    re.IGNORECASE
)

PATTERNS = {
    "patient_id": re.compile(r"\bPT[0-9]{6}\b"),
    "date":     re.compile(r"\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b"),
    "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s?(?:mg|mcg|g|ml|mL)\b", re.I),
    "bp": re.compile(r"\b[0-9]{2,3}\s*/\s*[0-9]{2,3}\s?mmHg\b", re.I),
    "age": re.compile(r"\b(?:[1-9][0-9]?|1[01][0-9]|120)\s?(?:years?|yrs?)\b", re.I)
}

ABBREVIATIONS = {
    "bp": "blood pressure", "hr": "heart rate", "rr": "respiratory rate",
    "sob": "shortness of breath", "htn": "hypertension",
    "dm": "diabetes mellitus", "uti": "urinary tract infection",
    "iv": "intravenous", "po": "by mouth", "prn": "as needed",
    "od": "once daily", "bd": "twice daily", "tds": "three times daily"
}

LEXICON = {
    "fractures": ("fracture", "N", "PL"),
    "patients": ("patient", "N", "PL"),
    "treated": ("treat", "V", "PAST"),
    "monitoring": ("monitor", "V", "PROG"),
    "higher": ("high", "ADJ", "COMP")
}

stemmer = PorterStemmer()

def process_record(text):
    tokens = TOKEN_RE.findall(text)
    normalised = []
    analyses = []
    for token in tokens:
        low = token.lower()
        normalised.append(ABBREVIATIONS.get(low, low))
        if low in LEXICON:
            analyses.append((token, LEXICON[low]))
    entities = {}
    for name, pattern in PATTERNS.items():
        entities[name] = pattern.findall(text)
    stems = [(t, stemmer.stem(t.lower())) for t in tokens if t.isalpha()]
```

```
    return tokens, normalised, entities, analyses, stems

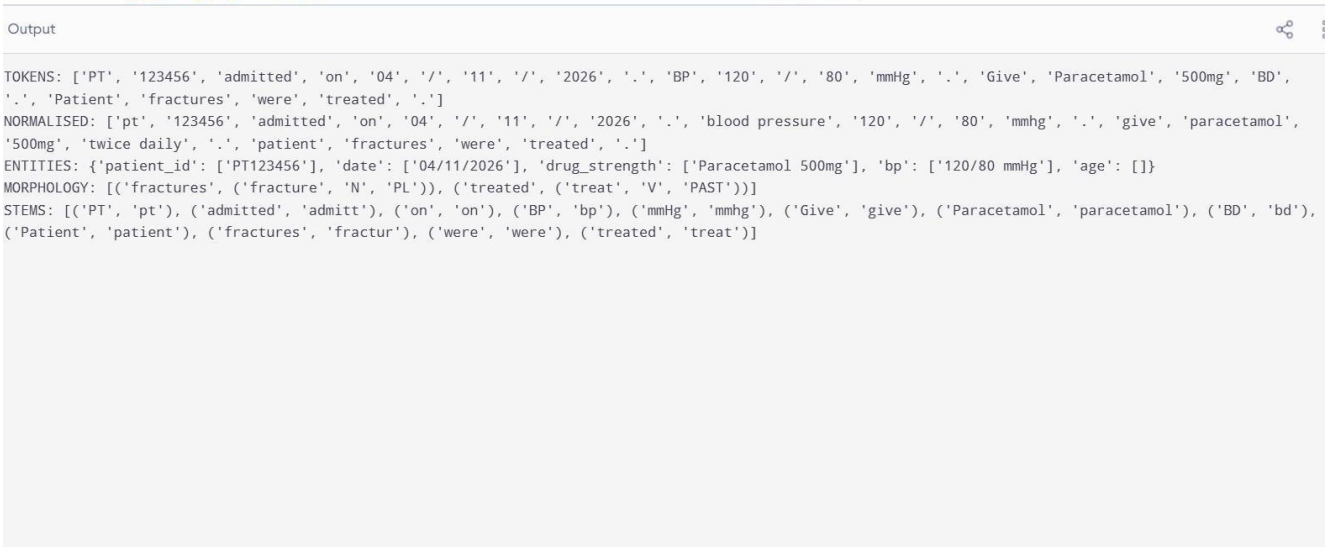
sample = ("PT123456 admitted on 04/11/2026. BP 120/80 mmHg. "
          "Give Paracetamol 500mg BD. Patient fractures were treated.")
tokens, normalised, entities, analyses, stems = process_record(sample)

print("TOKENS:", tokens)
print("NORMALISED:", normalised)
print("ENTITIES:", entities)
print("MORPHOLOGY:", analyses)
print("PORTER:", stems)
```

10.2 Test strategy

- Test every entity pattern with at least one valid and one invalid string.
- Test each DFA with accepted and rejected strings and compare with the regular-expression result.
- Test optional-hyphen NFA paths and the subset-converted DFA.
- Test regular, irregular and ambiguous morphological forms.
- Compare hand-calculated Porter examples with the NLTK implementation.

10.3 Results reporting template



Measure runtime with time.perf_counter() on the same machine and corpus. Report Python and NLTK versions, hardware details and corpus size. A plot can show runtime against record length or analysis coverage against lexicon size; the graph must use measured values.

TASK 11 – Compare, Recommend and Defend the Normalisation Design

11.1 Comparison

Criterion	No normalisation	Porter stemming	Finite-state morphology
Recall	May miss inflected variants.	Often improves variant matching.	Can cover listed inflectional and selected derivational variants.
Precision	Exact matches can be precise; format variation causes misses.	May over-conflate unrelated words.	Can preserve features and irregulars; quality depends on lexicon/rules.
Coverage	No language resources required.	Broad suffix coverage.	Limited by lexicon and rule inventory.
Runtime	Low preprocessing cost.	Usually fast.	Depends on lexicon and transducer size.
Maintenance	Low.	Low to moderate.	Higher: affixes, exceptions and clinical terminology need review.

11.2 Recommendation

Deploy a hybrid, conservative design: use regular expressions and automata for structured entities; normalise safe formatting variations; use finite-state morphology for inflection and approved clinical derivations; and use Porter stemming only as a fallback for out-of-lexicon general English terms. Do not conflate clinically important opposites or negated forms.

The design should retain original text, record every normalisation decision, and allow clinicians to inspect why a record matched. The final accuracy values must be based on the 40-query evaluation. The highest recall alone is not sufficient: a false match between clinically different conditions can be harmful.

11.3 Manual effort and economic threshold

Manual effort includes collecting and de-identifying records, writing and reviewing patterns, building the 300-stem lexicon, listing at least 20 inflectional and 15 derivational affixes, annotating 200 forms and judging 40 queries. A statistical or learned method becomes attractive when the cost of maintaining exceptions and regional vocabulary exceeds the cost of collecting representative labelled examples and validating a model. Human clinical review remains necessary.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

12.1 A. Impact analysis

The new sources introduce OCR noise and Romanised regional-language/English code-mixing. The original design assumed clean typed text, a mostly closed English vocabulary and exact matching. Those assumptions no longer hold.

New input property	Original component affected	Likely failure
0/O, 1/l, rn/m confusion	Regex and automata	Identifiers, dates and drug names may fail exact validation.
Line-break hyphenation	Tokenizer	One word may be split into fragments.
Lost diacritics	Lexicon and morphology	Regional or borrowed words may not match stored entries.
Out-of-lexicon words	FST	No analysis path is found.
Romanised code-mixing	English-only rules	English morphology cannot analyse regional suffixes or particles.
At least 20% noisy/OOV/non-English tokens	All exact-match stages	Coverage decreases and false negatives increase.

A 20% noisy-token estimate does not mean every metric falls by exactly 20%. Some noise is harmless, some affects multiple tokens, and some creates false positives. Estimate the actual degradation by running the same labelled records through the clean and noisy pipelines.

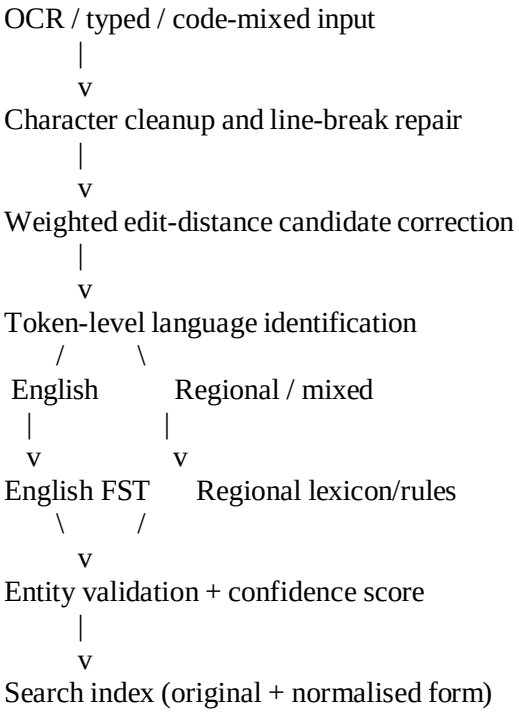
12.2 B. Evaluate the original design on new records

Record type	Synthetic input	First likely failure
OCR	PT12O456: fract ure noted; BP 12O/80 mmHg.	Identifier and BP patterns fail because O is confused with zero; split word is not in the lexicon.
Code-mixed	patient ku fever irundhuchu, paracetamol 500mg kuduthen	English morphology cannot analyse regional-language tokens; English drug-strength pattern may still match.
Combined	PT12345l admitted; patient-ku fever irundhuchu; dose 500mg.	Identifier fails due to l/1 confusion; code-mixed tokens remain OOV; some structured values still match.

For a reproducible evaluation, prepare at least three labelled noisy records and calculate pattern precision/recall, analyser coverage and retrieval F1. Compare these values with the clean-corpus baseline using the same queries and relevance judgements.

12.3 C. Adaptation or redesign

Decision: MODIFY the original design rather than discard the finite-state layer. Keep exact validation for clean inputs, but add a noise-tolerant front end and language-aware fallback.



- Add weighted edit-distance correction for likely OCR confusions such as O/0, l/1 and rn/m. Corrections should be suggestions with confidence scores, not silent replacements.
- Repair words split at line endings using a conservative hyphenation rule and preserve the original line text.
- Add token-level language identification to separate English, regional-language and uncertain tokens.
- Extend the lexicon with frequently used regional stems and common Romanised variants collected from approved records.
- Retain exact regular expressions for high-confidence fields, and add tolerant candidate matching only when the exact pattern fails.
- Store original text, corrected candidate, confidence and applied rule for auditability.

12.4 C. Expected trade-offs

Measure — Expected effect of modification — Risk/control

• Coverage — Increases for OCR variants and code-mixed vocabulary. — Avoid accepting low-confidence corrections as facts.

• Precision — May decrease if fuzzy matching is too permissive. — Use thresholds, context and clinician review.

• Runtime — Increases due to candidate generation and language identification. — Apply fuzzy correction only to failed or uncertain tokens.

• Annotation effort — Increases because regional forms and OCR variants need labels. — Prioritise frequent terms and high-impact clinical entities.

12.5 D. Evidence-based defence

The modified design is justified because the original exact finite-state layer remains valuable for clean structured data, but it cannot handle unlimited spelling variation or unseen regional morphology by itself. A weighted correction stage addresses character noise; token-level language identification routes mixed tokens; and expanded finite-state resources handle recurring vocabulary. The system should continue to expose uncertainty and preserve source text.

Remaining limitations include unseen languages, highly ambiguous abbreviations, semantic interpretation, negation and complex discourse. No amount of finite-state rule expansion alone provides complete clinical understanding. Human review and, where appropriate, statistical language models are needed for uncertain semantic cases.

Conclusion

This assignment presents a staged design for morphology-aware hospital record search. Regular expressions and finite automata recognise structured fields; tokenisation and normalisation reduce formatting variation; morphology relates surface forms to stems and grammatical features; and Porter stemming provides a lightweight fallback. The day-4 challenge shows that OCR noise and code-mixing require a modified, confidence-aware pipeline. The final deployment decision should be confirmed using a documented corpus, annotated examples and clinician relevance judgements.